IPTO

System and development documentation

2026-04-16

Contents

1 Introduction **7**

2 System Overview **8**

2.1 Shared architectural ideas 9

2.2 Database as the canonical persistent model 10

2.3 GraphQL SDL as configuration and runtime contract 10

2.4 Implementation surfaces 10

2.5 How to read the rest of the manual 11

3 Database **12**

3.1 Overview 12

3.2 Unit identity and versioning 14

3.3 Attribute catalog 14

3.4 Attribute occurrences and version ranges 15

3.5 Typed vector storage 15

3.6 Templates and record structure 16

3.7 Relations, external associations, locks and log 16

3.8 Bootstrap and deployment 17

3.9 Ingestion and extraction flow 17

3.10 A typical persistence round-trip 17

3.11 Observations 18

4 GraphQL SDL Configuration **19**

4.1 Top-level loading flow 19

4.2 Directive vocabulary 19

4.3 Datatype registry 20

4.4 Attribute registry 20

4.5 Templates and records 20

4.6 Operation roots and runtime bindings 20

4.7 How operation auto-wiring actually works . . . 21
4.8 The runtime wiring lifecycle . . . 21
4.9 Inference when `@ipto(operation:...)` is omitted . . . 22
4.10 Why some operations are rejected as ambiguous . . . 22
4.11 The `RuntimeOperation` enum . . . 23
4.12 Explicit binding always wins . . . 24
4.13 Custom and manual wiring . . . 24
4.14 What auto-wiring does and does not cover . . . 25
4.15 Reconciliation with the repository catalog . . . 25
4.16 Runtime wiring . . . 25
4.17 Worked example . . . 26
4.18 Observations . . . 26

5 Java Implementation . . . 27

5.1 Module structure . . . 27
5.2 Bootstrapping and configuration . . . 27
5.3 Repository layer . . . 27
5.4 SQL statement management . . . 28
5.5 Search expression model . . . 28
5.6 Building search trees . . . 28
5.7 Translation into SQL . . . 29
5.8 Textual query representation . . . 30
5.9 Field resolution and typed leaves . . . 30
5.10 Left and right relation semantics . . . 31
5.11 Parsing pipeline . . . 31
5.12 GraphQL configuration and runtime . . . 32
5.13 A typical Java-side flow . . . 32
5.14 Testing . . . 33
5.15 Observations . . . 33

6 Erlang Implementation **34**

6.1 Overall structure 34

6.2 Facade and orchestration 34

6.3 Backend contract 35

6.4 Backend implementations 35

6.5 GraphQL and HTTP surfaces 35

6.6 Testing and completeness 36

6.7 Observations 36

7 Rust and Python **37**

7.1 Rust 37

7.2 PyO3 bridge 37

7.3 Build and packaging model 38

7.4 Backend abstraction 38

7.5 Repository service layer 38

7.6 GraphQL runtime scope 39

7.7 Python 39

7.8 Current parity position 39

8 Admin Web **40**

8.1 Role in the system 40

8.2 Structure 40

8.3 Integration points 40

8.4 Packaging 41

8.5 Observations 41

9 Development Notes **42**

9.1 Repository layout 42

9.2 Java build and test workflow 42

9.3 Database provisioning 42

9.4 Erlang workflow 43

9.5 Rust and Python workflow . . . 43
9.6 Admin web workflow . . . 43
9.7 SDL-driven development . . . 44
9.8 Observations . . . 44
10 Appendix: Developing Against IPTO from Java . . . 45
10.1 Start from SDL . . . 45
10.2 Bootstrapping repository and runtime . . . 45
10.3 Working through the general Java API . . . 46
10.4 What the SDL changes for Java development . . . 47
10.5 Using the GraphQL interface . . . 47
10.6 Text search through GraphQL . . . 48
10.7 Raw payload flows . . . 48
10.8 Custom operations versus built-in operations . . . 48
10.9 Practical development advice . . . 49
11 Appendix: Key Commands and Environment Variables . . . 50
11.1 Core commands . . . 50
11.2 Important PostgreSQL variables . . . 50
11.3 Important Neo4j variables . . . 50
11.4 GraphQL SDL-related settings . . . 51
11.5 Practical advice . . . 51
11.6 Example unit payload . . . 51

1 Introduction

IPTO is a metadata-oriented repository system built around a shared persistence model, a schema-driven configuration model, and several implementation surfaces. The core idea is that information is stored as versioned units composed from catalogued attributes, while external schema definitions are used to declare how those attributes are grouped into templates, records and operations.

The repository is not a single program in a single language. It combines:

- a relational database model shared across implementations,
- a GraphQL SDL layer that also acts as configuration input,
- a Java reference implementation,
- an Erlang OTP-based implementation,
- a Rust implementation that can be embedded into Python through PyO3.

This manual is therefore organized by subsystem and responsibility. It begins with the shared architectural foundation in Section 2, then covers the database and SDL layers in Sections 3 and 4, and finally describes the implementation surfaces and the practical development workflow around them.

2 System Overview

At a high level, IPTO can be understood as a layered system with three centers:

- a persistent catalog and storage model in the database,
- a schema/configuration layer expressed in GraphQL SDL,
- one or more runtime implementations that expose repository and GraphQL functionality.

The database and the SDL layer are the shared conceptual backbone. The language-specific implementations differ in style and runtime environment, but they all relate back to those shared definitions.

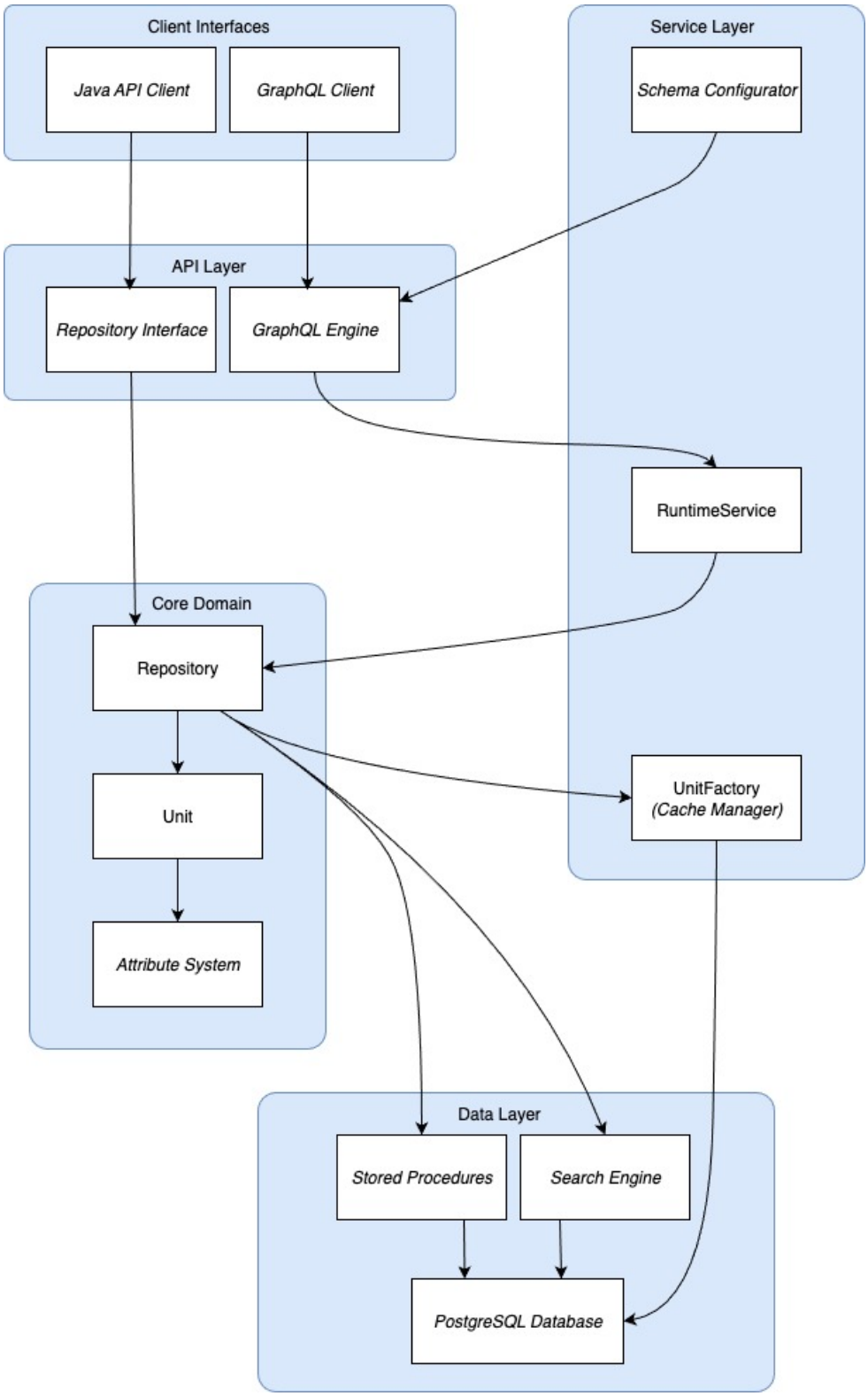

Figure 1: High-level view of the layered architecture: clients, API layer, core repository logic, service logic and data layer

2.1 Shared architectural ideas

Several design ideas recur across the repository:

Units as the primary entity Data is stored as units that belong to tenants and evolve through versions.
Attributes as catalogued metadata Units are described through attributes defined in a shared catalog rather than through bespoke tables per business object type.
Templates and records as shapes Higher-level structures are described through templates and nested records rather than being hard-wired into the storage schema.
Backend separation Repository operations are conceptually separated from backend-specific storage details, even if each implementation realizes that separation differently.

2.2 Database as the canonical persistent model

The relational schema is the canonical persistent representation of IPTO concepts. It defines:

- tenants, units and versions,
- the attribute catalog,
- typed value vectors,
- record and template metadata,
- relations, associations, locks and log entries.

This is why the database chapter, Section 3, comes early in the manual. The runtime layers make more sense once the storage model is understood.

2.3 GraphQL SDL as configuration and runtime contract

GraphQL SDL has a dual role in IPTO. It defines an API surface, but it also defines repository-facing metadata such as datatypes, attributes, records and templates. In other words, SDL is not just documentation for queries and mutations. It is also part of how the system is configured and aligned with the persistent catalog.

That dual role explains why the GraphQL SDL chapter, Section 4, also appears early. It is a shared architectural concern, not merely a feature of one runtime.

2.4 Implementation surfaces

The current implementations occupy different roles:

Java The most complete implementation and the current reference point for repository semantics, catalog reconciliation and GraphQL runtime behavior. See Section 5.
Erlang An OTP-oriented implementation with explicit backend abstraction, GraphQL support and HTTP service support. See Section 6.
Rust plus Python A Rust implementation that also exposes a Python-facing API through PyO3 and `maturin`. See Section 7.
Admin web A SvelteKit-based administration client for schema administration, tree browsing and search. See Section 8.

These implementations are not identical, and full feature parity should not be assumed. Still, they are all working against the same broader conceptual system.

2.5 How to read the rest of the manual

The remainder of the document follows the system from shared foundation to concrete implementation:

1. the database chapter explains the persistent model,
2. the GraphQL SDL chapter explains how schema definitions are interpreted and applied,
3. the implementation chapters describe how the different runtimes realize those concepts,
4. the development chapter explains how the repository is built, tested and evolved in practice, with the appendix collecting the most commonly used commands and variables.

This order is deliberate. In IPTO, storage and configuration are shared concepts, while the language-specific runtimes are realizations of those concepts.

3 Database

This section describes the persistence model that the rest of the system is built around. The database is not just a passive storage layer. It holds the canonical catalog of tenants, units, versions, attributes, templates, relations and associations, and it is designed so that the higher-level APIs can ingest and retrieve structured metadata without hard-coding a domain-specific table layout for each information type.

3.1 Overview

At the database level, *units* are the primary persistent entities. A unit belongs to a tenant, has a stable identity, carries status and audit information, and may exist in several versions over time. Each version is described by a set of attributes. Attribute definitions are global and live in a catalog, while actual attribute occurrences are stored per unit version.

The schema is implemented twice: once for PostgreSQL in `shared/db/postgresql` and once for DB2 in `shared/db/db2`. The two implementations share the same logical model and table groups, but differ where the database products require different data types, function mechanisms or deployment steps.

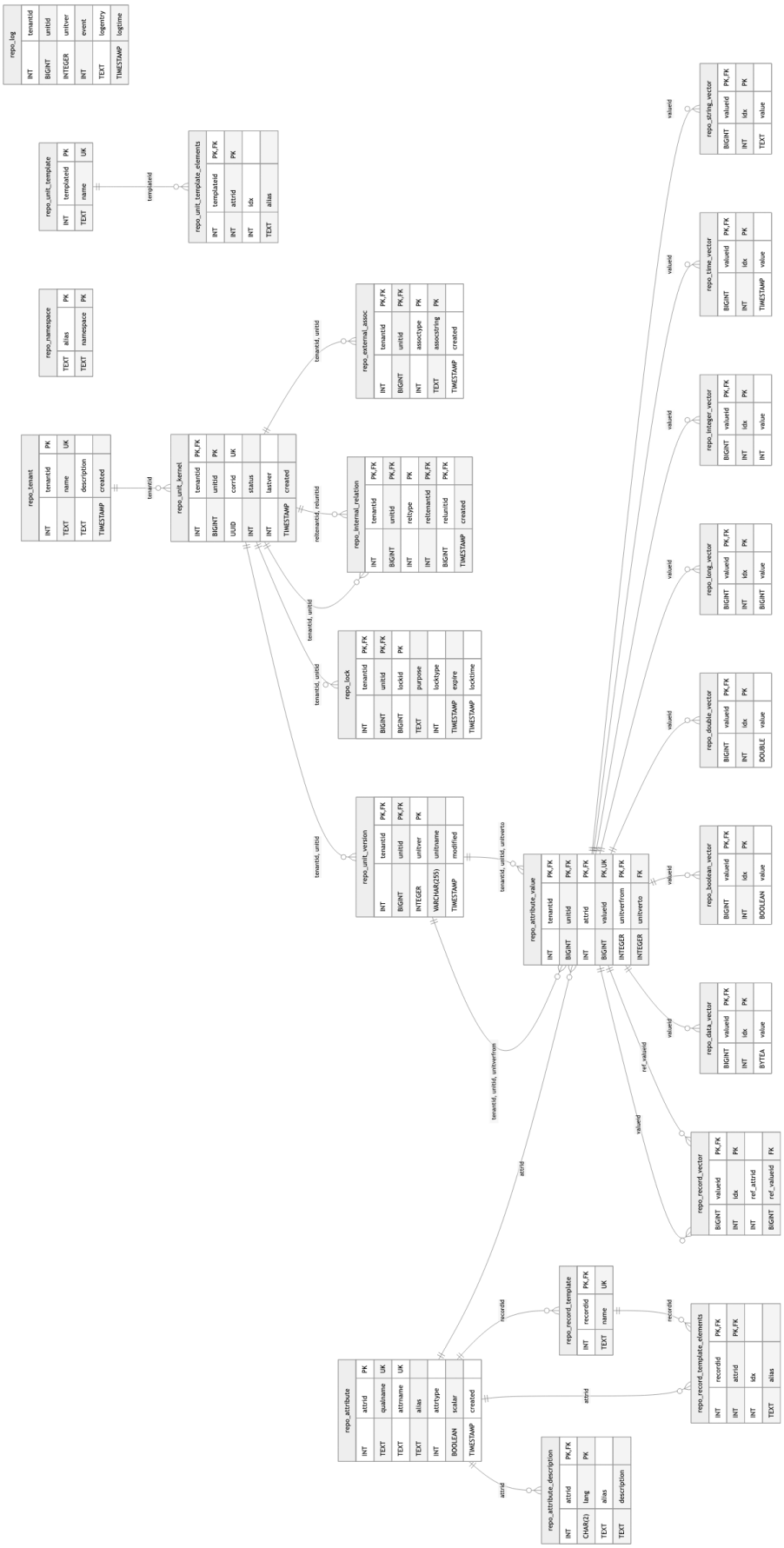

Figure 2: Repository schema overview derived from the Mermaid ER model in `doc/Model.md`

Figure 2 shows the full relational layout at schema level. The remainder of this chapter walks the same structure in conceptual groups so that the storage model is easier to understand than a table-by-table listing alone.

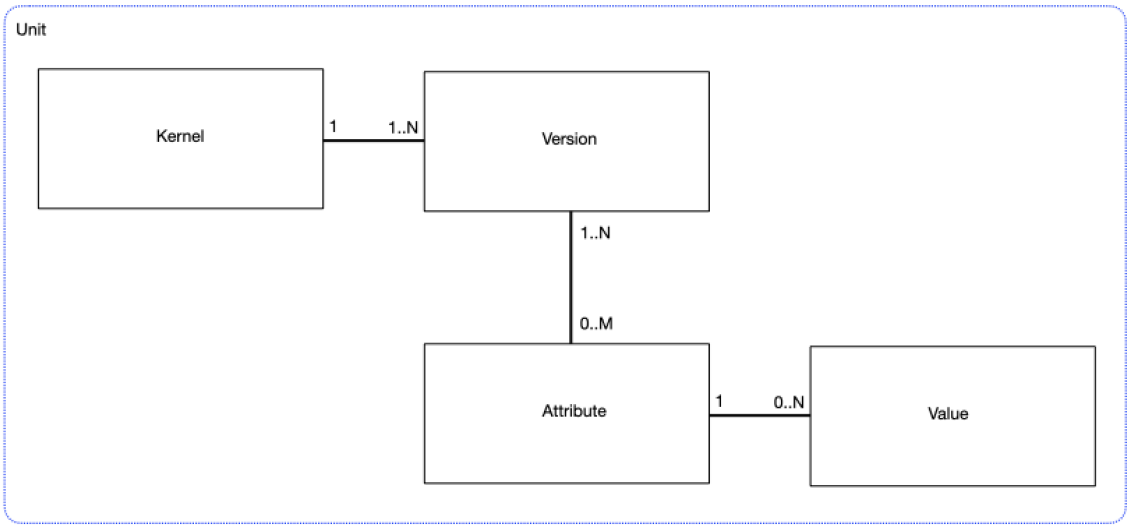

Figure 3: Core persistent concepts: a unit kernel, its versions, the attributes used in a version, and the corresponding value instances

3.2 Unit identity and versioning

The central unit lifecycle is represented by two tables:

`repo_unit_kernel` Holds the stable identity of a unit. The row contains the tenant id, the unit id, a correlation id, the current status, the latest version number and the original creation timestamp.

`repo_unit_version` Holds version-specific state for the same unit. Each row is keyed by `(tenantid, unitid, unitver)` and contains a version-local name and modification timestamp.

This split is important. It allows the system to keep a durable identity and current status in the kernel while storing historical changes in the version table. The latest version is found by following `repo_unit_kernel.lastver` to the matching row in `repo_unit_version`. Java repository queries use this directly when loading either a specific version or the latest one.

Tenant separation is explicit throughout the schema. Most tables that store unit-related information include `tenantid` as part of the key. This keeps multi-tenant isolation visible and queryable in the relational model rather than leaving it implicit in application code.

3.3 Attribute catalog

The database distinguishes between *what an attribute means* and *where an attribute value occurs*, analogous to, but simpler than, ISO/IEC 11179.

The catalog side consists primarily of the following tables:

`repo_namespace` Maps namespace aliases to full namespace URIs. The boot script preloads commonly used vocabularies such as Dublin Core, FOAF, DCAT and related namespaces.

repo_attribute Stores one row per known attribute. The row contains a qualified name, a local attribute name, an optional alias, the logical attribute type and whether the attribute is scalar or vector-valued.

repo_attribute_description Stores human-readable descriptions per language.

The attribute catalog is intentionally global. Units do not invent their own attribute definitions when data is written. Instead, repository code and the GraphQL-based configuration layer reconcile the incoming schema description against this catalog and populate missing definitions before units are ingested.

3.4 Attribute occurrences and version ranges

Actual use of an attribute in a unit version is represented by `repo_attribute_value`. This table is the bridge between the stable catalog and the typed value tables. Each row records:

- which tenant and unit the attribute belongs to,
- which catalog attribute is used,
- a generated `valueid`,
- the version interval where that value is valid, expressed as `unitverfrom` and `unitverto`.

The version interval is one of the key design decisions in the schema. A new version does not always require duplicating every value. In the PostgreSQL ingestion logic, `ingest_new_version_json` delegates to `repo_ingest_attributes_delta`. If an attribute occurrence is marked as unmodified, the existing value range is simply extended to cover the new version. If it is modified, a new `valueid` is created and fresh vector rows are written.

This gives the system snapshot semantics for units while still allowing physical reuse of unchanged values across successive versions.

3.5 Typed vector storage

The actual payload values are not stored directly in `repo_attribute_value`. Instead, each attribute occurrence points to one of several vector tables depending on datatype:

Table	Stored values
`repo_string_vector`	text values
`repo_time_vector`	timestamps
`repo_integer_vector`	integer values
`repo_long_vector`	64-bit integer values
`repo_double_vector`	floating point values
`repo_boolean_vector`	boolean values
`repo_data_vector`	binary payloads
`repo_record_vector`	references from a record value to nested child values

Each vector table uses `(valueid, idx)` as primary key. The `idx` column preserves order and also allows the same logical attribute to carry multiple values. In other words, arrays are a first-class part of the storage model rather than an application-side convention.

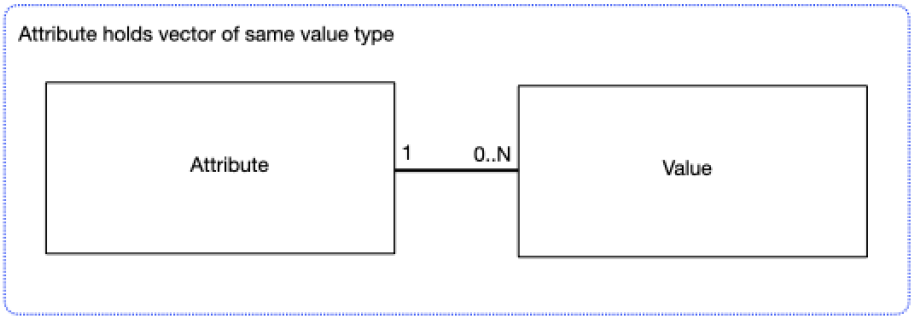

Figure 4: A single attribute occurrence may hold a vector of ordered values of one datatype

The special case is `repo_record_vector`. It does not store scalar payload data. Instead, it stores references to other attribute values, making nested record structures representable without abandoning the relational model. PostgreSQL uses recursive PL/pgSQL functions to walk nested JSON and populate these links. DB2 exposes equivalent ingest and extract operations through Java-backed stored procedures.

3.6 Templates and record structure

Two metadata table groups describe reusable structural shapes:

`repo_unit_template` and `repo_unit_template_elements` Describe the ordered set of attributes that make up a top-level unit template.

`repo_record_template` and `repo_record_template_elements` Describe nested record types and the ordered attributes that belong to each record.

These tables are not populated by ad hoc SQL in day-to-day use. They are synthesized from the GraphQL SDL configuration described later in this manual. The configuration layer reads directives such as `@template`, `@record` and `@use`, compares the SDL view with the catalog already present in the repository, and inserts or reconciles the corresponding catalog and template rows.

3.7 Relations, external associations, locks and log

Besides metadata content itself, the schema contains supporting tables for operational behavior:

`repo_internal_relation` Stores typed relations between two units in the repository.

`repo_external_assoc` Stores typed associations from a unit to an identifier in an external system.

`repo_lock` Stores repository locks with a purpose, a lock type and an optional expiry timestamp.

`repo_log` Stores audit-like event information per unit version.

These tables keep relationship management and coordination close to the unit model. They are also referenced by the Java repository layer through predefined SQL statements, for example when loading log entries, counting relations or deleting a unit and relying on cascading cleanup.

3.8 Bootstrap and deployment

Both database variants are provisioned from a small set of scripts:

`schema.sql` Creates the schema, tables, constraints and indexes.

`procedures.sql` Installs the ingest and extract routines.

`boot.sql` Loads initial tenants, root units and namespace mappings.

The PostgreSQL test setup script executes `schema.sql`, `procedures.sql` and `boot.sql` inside a Docker-hosted PostgreSQL instance. The default configuration used by the Java repository points at `jdbc:postgresql://localhost:5432/repo`.

The DB2 setup follows the same logical sequence, but with one additional deployment step. Before the DB2 procedures are created, `load-udf.sql` installs the Java archive that implements the stored procedures. This is why the DB2 procedure definitions refer to Java entry points such as `org.gautelis.ipto.repo.udpf.Routines.ingestNewUnit`.

3.9 Ingestion and extraction flow

On PostgreSQL, unit persistence is intentionally centered on database procedures rather than distributing all persistence logic into application code. The main procedure pair is:

`ingest_new_unit_json` Creates a new kernel row, creates the first version row, and recursively ingests the attribute tree.

`ingest_new_version_json` Advances the kernel version, inserts the new version row, and applies delta-based attribute ingestion.

The recursive work is performed by `repo_ingest_attributes` and `repo_ingest_attributes_delta`. These routines inspect each attribute occurrence, create a row in `repo_attribute_value`, route primitive payloads into the matching vector table, and create `repo_record_vector` links for nested records.

The inverse operation is exposed through `extract_unit_json`. This procedure reconstructs a JSON representation of a unit by combining:

- kernel and version metadata,
- the active attribute occurrences for the requested version,
- the corresponding primitive vector content or nested record structure.

This means the relational schema is not only the persistent format. It is also the source from which the external JSON unit representation can be recreated.

3.10 A typical persistence round-trip

The typical storage round-trip in the current design looks like this:

1. a runtime constructs or receives a unit as JSON,
2. the unit header is written into `repo_unit_kernel` and `repo_unit_version`,
3. each attribute occurrence becomes a `repo_attribute_value` row,

4. primitive values are placed in the matching vector table,
5. record-valued attributes recursively create child attribute values and parent-child links in `repo_record_vector`,
6. later retrieval reconstructs the unit by walking the same structures in reverse.

This flow explains why the schema can remain generic while still supporting rich nested metadata structures. The database stores normalized relational state, while the procedures provide the translation boundary between that state and the JSON-shaped unit payloads used by higher layers.

3.11 Observations

The database design is deliberately hybrid:

- relational tables are used for identity, integrity, indexing and joins,
- vector tables provide ordered multi-value support for primitive datatypes,
- record vectors allow nested structures without creating one table per business object,
- version ranges make unchanged content reusable across successive unit versions,
- the catalog and template tables make the model configurable from external schema descriptions rather than fixed in compiled code.

That combination is what makes the rest of the system possible: GraphQL SDL can define shapes, the Java repository can persist and search them, and the alternative runtime implementations can target the same underlying conceptual model.

4 GraphQL SDL Configuration

IPTO uses GraphQL SDL for two related but distinct purposes. First, SDL defines the executable GraphQL interface exposed at runtime. Second, the same SDL is used as a configuration source from which the system derives repository datatypes, attributes, records, templates and operation bindings.

The central entry point is the Java class `org.gautelis.ipto.graphql.configuration.Configurator`. Its `load(...)` method parses SDL, derives a GraphQL-side model, loads the current catalog from the repository, reconciles the two views, and finally builds the executable GraphQL schema.

4.1 Top-level loading flow

The configuration flow is intentionally staged:

1. parse SDL into a GraphQL `TypeDefinitionRegistry`,
2. derive the SDL viewpoint: datatypes, attributes, records, templates, unions and operations,
3. read the current repository viewpoint from the database catalog,
4. reconcile the differences and populate missing catalog entries,
5. build runtime wiring and create the executable GraphQL schema.

This makes the SDL the authoritative description of the intended logical model, while the database remains the durable store of the resulting catalog and template metadata.

4.2 Directive vocabulary

The shipped schema file `implementations/java/quarkus-app/src/main/resources/ipto.graphqls` defines a small directive vocabulary used by the loader:

`@datatypeRegistry` Marks an enum whose values define basic datatypes.

`@datatype(id:Int)` Assigns the numeric repository datatype id used internally by IPTO.

`@attributeRegistry` Marks an enum whose values define repository attributes.

`@attribute(...)` Declares the datatype, cardinality, repository name, URI and description of an attribute.

`@template(name:String)` Marks an object type as a top-level unit template.

`@record(attribute:Attributes!)` Marks an object type as a nested record bound to a record attribute.

`@use(attribute:Attributes!)` Binds a field in a template or record to a declared repository attribute.

`@ipto(operation:String!)` Binds a GraphQL operation field to a known runtime operation.

The SDL layer is therefore not generic GraphQL schema usage. It is GraphQL plus a repository-specific annotation vocabulary that drives catalog synthesis.

4.3 Datatype registry

Basic datatypes are derived from an enum annotated with `@datatypeRegistry`. In the current schema this is the `DataTypes` enum. Values such as `STRING`, `TIME`, `INTEGER`, `LONG`, `DOUBLE`, `BOOLEAN`, `DATA` and `RECORD` are each annotated with a numeric id via `@datatype(id:...)`.

The loader validates these ids against the internal Java enum `AttributeType`. This is an important constraint: SDL cannot invent arbitrary numeric datatypes. It can only refer to the officially supported storage types already known to the repository implementation.

4.4 Attribute registry

Repository attributes are derived from an enum annotated with `@attributeRegistry`. Each enum value becomes a logical attribute definition. The `@attribute` directive contributes the important parts:

- the underlying datatype,
- whether the attribute is array-capable,
- the repository attribute name,
- the qualified URI or namespace-backed identifier,
- an optional human-readable description.

If the `name` argument is omitted, the enum value name is used. If the `uri` argument is omitted, the repository name is reused as fallback. The resulting attribute shapes are later compared with the rows in the database tables `repo_attribute` and `repo_attribute_description`.

This is the bridge between the GraphQL schema vocabulary and the database catalog described in Section 3.

4.5 Templates and records

Top-level domain objects are represented as GraphQL object types. The loader distinguishes operation root types from ordinary domain types, and then interprets the remaining object types in one of two ways:

`@template` Declares that an object type describes a top-level unit template.

`@record` Declares that an object type describes a reusable nested record.

For both templates and records, the actual field structure is derived from fields annotated with `@use(attribute:...)`. That directive allows the field name used in SDL to differ from the underlying repository attribute name. If `@use` is omitted, the loader attempts to resolve the field name directly against the attribute registry.

For templates, the resulting metadata is persisted in `repo_unit_template` and `repo_unit_template_elements`. For records, the corresponding metadata is persisted in `repo_record_template` and `repo_record_template_elements`.

4.6 Operation roots and runtime bindings

Operation roots are resolved from the GraphQL `schema {...}` declaration when present, otherwise the defaults `Query`, `Mutation` and `Subscription` are assumed. The loader derives one operation shape per field in those root types.

Each operation shape contains:

- the operation category: query, mutation or subscription,
- the operation name,
- its input parameters,
- the result type,
- an optional runtime binding from `@ipto(operation:...)`.

At present, subscriptions are recognized structurally, but runtime subscription support is not yet implemented. That means the SDL may describe subscription fields, but operational support remains incomplete.

4.7 How operation auto-wiring actually works

The important detail is that IPTO does not treat every query or mutation field as application-specific code that must be wired by hand. Instead, the Java GraphQL runtime tries to bind operation fields to a small set of built-in repository behaviors.

That binding happens in two stages:

1. the SDL loader derives a `GqlOperationShape` for each field in the operation root types,
2. the runtime wiring step decides whether that shape can be wired automatically, must be wired explicitly through `@ipto(operation:...)`, or must be supplied by custom Java code.

The derived operation shape is intentionally small. It contains:

- the GraphQL root type where the field lives, for example `Query` or `Mutation`,
- the field name,
- the category (`QUERY`, `MUTATION` or `SUBSCRIPTION`),
- the ordered parameter list with GraphQL type information,
- the declared output type,
- the optional `RuntimeOperation` value parsed from `@ipto(operation:...)`.

This means the auto-wiring logic is based on operation shape, not on the field name alone. A field called `yrkan` is not special because of its name. It is special because its arguments and result type make it compatible with one of the known runtime operations.

4.8 The runtime wiring lifecycle

The full lifecycle in the Java implementation is:

1. `Configurator.load(...)` parses SDL and determines which object types are operation roots.
2. `Operations.derive(...)` creates one `GqlOperationShape` per query, mutation or subscription field.
3. If the field carries `@ipto(operation:...)`, the directive argument is parsed into the enum `RuntimeOperation`.
4. `OperationBindingValidator` checks whether an operation without an explicit binding is safe to infer.

5. `RuntimeService.wire(...)` delegates operation wiring to `RuntimeOperators.wireOperations(...)`.
6. For each operation:
   - if the enum value is `CUSTOM` or `MANUAL`, auto-wiring is skipped,
   - otherwise a GraphQL `DataFetcher` is installed automatically,
   - at execution time that fetcher delegates to `RuntimeOperationInvoker.invokeOperation(...)`.
7. `RuntimeOperationInvoker` then either:
   - dispatches directly according to the explicit enum value, or
   - infers the built-in repository behavior from the operation shape when no enum value is present.
8. After the built-in wiring is installed, the optional `operationsWireBlock` callback may add or override data fetchers programmatically.

The enum in `org.gautelis.ipto.graphql.model.RuntimeOperation` is the explicit contract between SDL and the Java runtime dispatcher.

4.9 Inference when `@ipto(operation:...)` is omitted

If no explicit runtime operation is supplied, the runtime falls back to shape-based inference. This is intentionally narrow. The Java implementation only infers a built-in operation when the argument structure is sufficiently recognizable.

The current inference rules are:

`Query` with one parameter named `id` Internally, the runtime code has handlers for two different identification shapes:

- `UnitIdentification` with `tenantId` and `unitId`, or
- a correlation-id carrier with `corrId`.

If execution ever reaches shape-based dispatch, the result type decides between typed loading and raw-payload loading. However, SDL operations with a single `id` parameter are considered ambiguous during configuration and should therefore not rely on omission of `@ipto(operation:...)`. In practice, `id`-based operations are expected to declare the intended runtime operation explicitly.

`Query` with one parameter named `filter` The runtime interprets the argument as `Filter`. If the result type is `Bytes`, a raw search variant is used; otherwise the ordinary typed search variant is used.

`Mutation` with one parameter named `data` of type `Bytes` The runtime interprets the field as raw unit storage.

If none of these patterns match, execution fails with an error saying that there is no automatic runtime mapping for the operation.

This explains an important design tradeoff. Omitting `@ipto(operation:...)` can be convenient, but it is only safe when the field shape already implies one unambiguous built-in meaning.

4.10 Why some operations are rejected as ambiguous

The configuration layer does not wait until request execution to detect every ambiguity. Before the runtime is even built, `OperationBindingValidator` checks operations that have no explicit `@ipto(...)`

binding.

It rejects operations whose shape could reasonably mean more than one runtime behavior. The important cases are:

- a single parameter named `id`, because the runtime has support for both unit-id loading and correlation-id loading and the SDL should therefore state explicitly which one is intended,
- a single parameter named `filter` returning `Bytes`, because that could mean raw unit search or raw payload search,
- any other single-parameter operation whose shape is not one of the recognized safe defaults.

In those cases the manual intent is expected to be explicit: add `@ipto(operation:"...")` so the SDL states exactly which built-in behavior is wanted.

The validator therefore serves two purposes:

- it prevents runtime surprises,
- it documents that operation fields are part of the repository configuration contract.

4.11 The `RuntimeOperation` enum

The enum `org.gautelis.ipto.graphql.model.RuntimeOperation` names the built-in dispatch targets understood by the Java runtime. Each value selects a concrete method on the runtime service layer.

`LOAD_UNIT` Load one typed unit by `tenantId` and `unitId`. The GraphQL field is expected to accept an `id` argument that can be coerced to `UnitIdentification`. The result is then projected through the configured GraphQL template and record shapes.

`LOAD_UNIT_RAW` Load one unit as raw repository payload bytes by `tenantId` and `unitId`. This bypasses typed field projection and exposes the stored generic payload.

`LOAD_BY_CORRID` Load one typed unit by correlation id instead of by internal unit id. In the current Java implementation, the GraphQL `id` argument is modeled as carrying only `corrId`.

`LOAD_RAW_PAYLOAD_BY_CORRID` Load raw payload bytes by correlation id. In the current Java implementation, the GraphQL `id` argument is likewise modeled as carrying only `corrId`. This is the correlation-id counterpart to `LOAD_UNIT_RAW`.

`SEARCH` Execute a repository search using the `Filter` input object and return typed GraphQL units. The search expression in `filter.where` is delegated into the repository search subsystem, not implemented separately in GraphQL.

`SEARCH_RAW` Execute a search using `Filter` and return raw units as bytes. This is the raw counterpart to `SEARCH`.

`SEARCH_RAW_PAYLOAD` Execute a search using `Filter` and return raw payloads rather than typed unit projections. This exists because there is more than one meaningful "raw" search surface, which is exactly why `filter -> Bytes` is considered ambiguous without an explicit binding.

`STORE_RAW_UNIT` Store a raw unit supplied as `Bytes`. The GraphQL mutation is expected to accept a `data` argument containing the payload to persist.

CUSTOM Declare that the SDL field is intentional, but that the built-in auto-wiring must not handle it. The field should be wired through the application-supplied `operationsWireBlock` or equivalent Java runtime code.

MANUAL Semantically very close to `CUSTOM`: it tells the runtime to skip built-in auto-wiring and requires the embedding code to provide the data fetcher. In other words, both values act as "do not auto-wire this field" markers.

In the current implementation, `CUSTOM` and `MANUAL` are treated the same by the auto-wiring code. Both cause the operation to be skipped during automatic GraphQL wiring, and both cause the runtime invoker to reject accidental built-in dispatch if they somehow reach it. The distinction is therefore documentary rather than behavioral at present.

4.12 Explicit binding always wins

When `@ipto(operation:...)` is present, the runtime does not try to infer a meaning from the field shape. The explicit enum value takes precedence.

This is an important rule because it makes the SDL resilient to future refactoring. For example, a field named `unitRaw(id: UnitIdentification!): Bytes` might currently be inferable from shape alone. But if the intent is part of the contract and should remain stable even if internal inference heuristics evolve, the explicit annotation is the safer choice:

```
type Query {
    unitRaw(id: UnitIdentification!): Bytes  @ipto(operation: "LOAD_UNIT_RAW")
}
```

The same applies to correlation-id based lookups and the two raw-search variants. Whenever the shape could be read in more than one way, the explicit enum value should be preferred.

4.13 Custom and manual wiring

Not every GraphQL field corresponds to a built-in repository primitive. Some fields need application-specific behavior, domain-specific validation, tenant-aware context handling, or orchestration that lives above the repository layer.

That is why `Configurator.load(...)` accepts an optional `operationsWireBlock`. This callback receives:

- the GraphQL `RuntimeWiring.Builder`,
- the `RuntimeService`,
- the `Repository`.

The intended usage is:

1. declare the field in SDL,
2. mark it with `@ipto(operation:"CUSTOM")` or `@ipto(operation:"MANUAL")` when you want to document that it is not a built-in operation,
3. attach the real `DataFetcher` in Java through the wiring callback.

This gives a clean split:

- SDL still defines the visible GraphQL contract,
- built-in operations remain declarative,
- genuinely custom behavior is kept in code where it belongs.

4.14 What auto-wiring does and does not cover

It is also useful to separate operation auto-wiring from the rest of the GraphQL runtime wiring:

- record-field wiring is automatic and maps projected GraphQL fields back to repository attributes,
- union wiring is automatic and resolves which concrete record type a stored value represents,
- operation-field wiring is automatic only for the built-in operation shapes and enum values described above,
- subscription fields are recognized, but operational runtime support is still scaffold-only.

So "auto-wiring" in IPTO does not mean that arbitrary application logic is inferred from SDL. It means that a deliberately small repository-oriented set of query and mutation patterns can be connected automatically to the Java runtime service layer.

4.15 Reconciliation with the repository catalog

After the SDL viewpoint has been derived, the configuration layer loads the current catalog viewpoint from the repository. This includes:

- the supported datatypes,
- the stored attribute definitions,
- stored record templates,
- stored unit templates.

The `CatalogReconciler` then compares the SDL-derived model with the catalog model. The purpose is not just validation. It is also to create missing catalog entities so that subsequent repository operations can store unit data that conforms to the SDL.

This is the operational meaning of using SDL to "prep" the system. The SDL does not merely document the domain. It actively drives the population and alignment of the repository catalog.

4.16 Runtime wiring

Once reconciliation is complete, `Configurator.load(...)` constructs the runtime GraphQL schema. It registers custom scalars such as `Long`, `DateTime` and `Bytes`, delegates repository-backed behavior to the runtime service layer, and then optionally allows additional operation wiring to be attached by the embedding application.

The result is that one SDL file plays three roles at once:

1. it defines the repository-facing metadata model,
2. it defines the GraphQL query and mutation surface,
3. it defines how those two views are stitched together at runtime.

4.17 Worked example

The pattern becomes clearer with a very small example. A schema may define an attribute registry entry, a record type and a template type along the following lines:

```
enum DataTypes @datatypeRegistry {
    STRING @datatype(id: 1)
    RECORD @datatype(id: 99)
}

enum Attributes @attributeRegistry {
    title @attribute(datatype: STRING, array: false, name: "dcterms:title")
    personName @attribute(datatype: STRING, array: false, name: "example:personName")
    person @attribute(datatype: RECORD, array: false, name: "example:person")
}

type Person @record(attribute: person) {
    name: String @use(attribute: personName)
}

type ExampleUnit @template(name: ExampleUnit) {
    title: String @use(attribute: title)
    owner: Person @use(attribute: person)
}
```

From this, the configuration layer can derive:

- datatype identities for `STRING` and `RECORD`,
- catalog entries for `dcterms:title`, `example:personName` and `example:person`,
- a record template for `Person`,
- a unit template for `ExampleUnit`.

Once those catalog entries exist, a stored unit can refer to the repository attributes by id and datatype in the generic JSON persistence format described by the repository layer.

4.18 Observations

The GraphQL SDL approach is central to how IPTO avoids hard-coding domain-specific storage models. Instead of creating new database tables for each business object type, the system derives catalog and template metadata from SDL and stores actual data through the generic unit, attribute and vector structures described in the database chapter.

That design also explains why the GraphQL configuration code is not just a thin schema parser. It is effectively a model compiler from SDL into repository metadata and runtime behavior.

5 Java Implementation

The Java implementation is currently the primary and most complete implementation of IPTO. It contains the repository abstraction, the SQL-backed persistence layer, the GraphQL configuration and runtime logic, and the integration tests that exercise the end-to-end behavior against the database.

5.1 Module structure

The Java code is split across several Maven modules under `implementations/java`:

`repo` The core repository implementation, including the model, persistence code, search query generation and database interaction.

`graphql` The GraphQL configuration and runtime layer.

`it` Integration tests that exercise the repository and GraphQL layers against a real database.

`quarkus-app` An application packaging of the runtime with a concrete GraphQL schema resource.

5.2 Bootstrapping and configuration

The repository is normally bootstrapped through `RepositoryFactory`. This factory loads:

- SQL statements from `sql-statements.xml`,
- database configuration from either `configuration_pg.xml` or `configuration_db2.xml`,
- optional action listeners via the plugin mechanism,
- the configured database adapter for search and backend-specific SQL generation.

Database access is pooled through HikariCP, and the factory applies prepared-statement cache settings suitable for high call rates. The database provider defaults to PostgreSQL unless the environment variable `IPTO_DB_PROVIDER` selects DB2.

5.3 Repository layer

The central class in the Java implementation is `org.gautelis.ipto.repo.model.Repository`. It is a high-level service that owns the main repository use cases rather than exposing raw SQL or raw table access.

At a high level, the repository layer is responsible for:

- creating units and assigning correlation ids,
- storing and retrieving units,
- managing status transitions,
- handling locks,
- handling relations and external associations,
- exposing catalog lookup operations,
- executing searches through the configured database adapter,

- giving controlled access to raw JDBC connections when needed.

The repository is therefore the main Java-side abstraction boundary between the rest of the application and the persistence model described in the database chapter.

5.4 SQL statement management

Not all SQL is embedded directly in Java classes. A substantial part of the CRUD-style SQL is stored in `implementations/java/repo/src/main/resources/org/gautelis/ipto/repo/sql-statements.xml`. These statements cover, among other things:

- unit lookup and deletion,
- attribute creation and inspection,
- lock operations,
- log retrieval,
- internal relation and external association access.

This provides a stable named-statement layer for the repository implementation. By contrast, the more dynamic search path is synthesized through the database adapter mechanism.

5.5 Search expression model

The Java implementation contains a dedicated search subsystem under `org.gautelis.ipto.repo.search.query`. Search requests are expressed as typed search trees using the sealed `SearchExpression` hierarchy rather than concatenating SQL fragments by hand.

The shape of the tree is deliberately small:

`LeafExpression` A single search predicate backed by one `SearchItem`.

`AndExpression` and `OrExpression` Binary nodes that combine two child expressions.

`NotExpression` Unary negation around another expression.

At the leaves, the actual domain constraint is carried by typed `SearchItem` implementations. These distinguish between:

- unit-level predicates such as tenant, status, unit name and timestamps,
- attribute-level predicates such as string, time, integer, long, double or boolean attribute values,
- relation and association predicates.

This means the search tree is not just a syntactic structure. Each leaf already knows whether it constrains unit-kernel columns, version fields, attribute values, relations or associations, and it also carries the operator and typed value involved in that predicate.

5.6 Building search trees

Search expressions are normally assembled through `QueryBuilder`. That helper exposes methods such as `assembleAnd(...)`, `assembleOr(...)`, `assembleAndNot(...)` and a set of typed constraint

builders such as `constrainToSpecificStatus(...)`, `constrainToSpecificTenant(...)` and `constrainOnValueEQ(...)`.

Conceptually, a search can therefore be built as an abstract syntax tree:

```
SearchExpression expr =
    QueryBuilder.assembleAnd(
        QueryBuilder.constrainToSpecificTenant(tenantId),
        QueryBuilder.assembleAnd(
            QueryBuilder.constrainToSpecificStatus(Unit.Status.EFFECTIVE),
            StringAttributeSearchItem.constrainOnEQ("dcterms:title", "Example")
        )
    );
```

The important point is that this object tree remains backend-neutral. It captures repository search intent in a typed form before any SQL dialect decisions are made.

5.7 Translation into SQL

The database adapter translates that search tree into backend-specific SQL. The common logic lives in `CommonAdapter`, while concrete backends such as `PostgresAdapter` and `DB2Adapter` provide database-specific behavior.

`CommonAdapter` first walks the tree and separates leaf predicates into two broad groups:

- unit constraints, which can be evaluated directly against unit kernel/version columns,
- non-unit constraints, such as attribute, relation and association predicates, which are given generated labels such as `c1`, `c2`, `c3`.

That separation matters because attribute-like predicates are not compiled into one large flat `WHERE` clause. Instead, each labeled leaf can produce its own SQL fragment, and the surrounding boolean structure of the AST is preserved by combining those fragments with SQL set operations.

For example, in set-operation mode:

- `AND` between two labeled leaves becomes `INTERSECT`,
- `OR` becomes `UNION`,
- `NOT` wraps the generated subexpression as a negated set clause.

Each `LeafExpression` is itself responsible for producing the appropriate SQL fragment for its search item and search strategy. For attribute predicates, that means joining `repo_attribute_value` to the correct typed vector table, resolving the attribute name to an attribute id, applying the operator to the typed value column, and constraining the result to the effective unit-version interval. For unit predicates, the leaf instead emits a direct comparison against the corresponding unit or version column.

The result is that Java search compilation is tree-driven rather than string-driven. The adapter does not try to parse SQL-like text. It compiles an already typed search AST into executable SQL while preserving the boolean structure of the original query.

5.8 Textual query representation

The Java search subsystem also supports a textual query form. This is useful when search criteria originate from user-facing filters, saved searches or API input rather than from direct Java construction through `QueryBuilder`. The textual form is parsed by `SearchExpressionQueryParser` and translated into the same `SearchExpression` tree described above.

This means there is only one real search model in the implementation. Programmatic construction and textual parsing both converge on the same AST, and the adapter layer compiles both through the same SQL-generation path.

The query language is intentionally compact:

- predicates have the form `field operator value`,
- boolean composition uses `AND`, `OR` and `NOT`,
- parentheses can be used for grouping,
- supported comparison operators include =, ==, !=, <>, <, <=, > , >= and `LIKE`,
- values can be strings, numbers, booleans or identifiers.

Operator precedence follows the parser grammar:

- `NOT` binds most tightly,
- `AND` binds more tightly than `OR`,
- parentheses override the default grouping.

A representative query can therefore look like:

```
tenantid = 17 AND status >= EFFECTIVE
AND (dcterms:title LIKE "example*" OR unitname = "example-unit")
```

5.9 Field resolution and typed leaves

When a textual query is parsed, each predicate field is resolved into one of the same typed search items used by programmatic search construction.

The parser first checks whether the field refers to one of the built-in unit fields, such as:

- correlation id,
- unit name,
- tenant id,
- status,
- created timestamp.

If the field is not a unit field, the parser resolves it as an attribute name or alias through the repository-backed attribute resolver. That resolution step yields both the canonical attribute name and its datatype, which allows the parser to build the correct `AttributeSearchItem` variant. A textual predicate on a time-valued attribute therefore becomes a time search leaf; a predicate on an integer-valued attribute becomes an integer search leaf, and so on.

The parser also recognizes relation and association predicates through qualified field prefixes such as `relation`, `relations`, `rel`, `association`, `associations` and `assoc`. Those predicates are mapped directly to the corresponding left/right relation or association search items. For those cases, only equality is accepted, which matches the underlying repository semantics.

5.10 Left and right relation semantics

The terms *left* and *right* are directional names taken from the binary relation model. In practice they should be read from the point of view of the unit being searched for.

For internal relations, IPTO stores one unit on the left side of the relation row and one unit on the right side. A left-relation search asks for units that stand on the left side when the right-side unit is already known. A right-relation search asks for units that stand on the right side when the left-side unit is already known.

For the built-in `PARENT_CHILD_RELATION`, that gives a natural hierarchy interpretation:

- a left-relation query retrieves children of a known parent,
- a right-relation query retrieves parents of a known child.

That is why left relations are a natural fit for optional directory-like hierarchies. If one unit represents a folder or container, and the contained units are related to it through `PARENT_CHILD_RELATION`, then a left-side relation query can enumerate the immediate children of that container. Repeating that step level by level yields a tenant-scoped browse tree without introducing a separate directory table into the schema.

The important design point is that this hierarchy is optional and repository-level rather than structural in the storage kernel. Units do not have to belong to a tree. A tenant may use parent-child relations to maintain one or more browsing hierarchies, while other units may remain outside any hierarchy at all.

Associations follow the same directional naming, but they point from a unit to an external identifier string rather than to another unit. A right association expresses that the current unit points to some external object. A left association finds units that are pointed to by a given external identifier. In other words:

- relations connect units to units inside the repository,
- associations connect units to references outside the repository.

This shared left/right convention keeps the parser and search model uniform. Once a field has been resolved as relation- or association-based, the resulting search leaf still fits into the same `SearchExpression` tree and is compiled by the same adapter pipeline as the other predicates.

One subtle but useful behavior is that string equality can be normalized into `LIKE` when the parsed string value carries pattern syntax. This allows the textual form to stay compact while still producing the correct operator at the AST level.

5.11 Parsing pipeline

The textual form is parsed with an ANTLR grammar under `implementations/java/repo/src/main/antlr4/org/gautelis/ipto/repo/search/query/SearchExpression.g4`. The pipeline is:

1. tokenize and parse the query string,

2. build a boolean expression tree with the grammar visitor,
3. resolve fields into unit, attribute, relation or association search items,
4. convert values into typed Java representations such as `Instant`, integer, long, double, boolean or string,
5. return a fully typed `SearchExpression` ready to be wrapped in `UnitSearch` and compiled by the selected adapter.

So the textual query language is not a separate execution path. It is an alternate frontend for creating the same internal search AST that Java code can also construct directly.

Among the current implementations, Java has the richest textual-query frontend. The Erlang and Rust implementations also support textual or symbolic search forms, but those are currently simpler and should not be assumed to match the Java parser feature for feature.

This split is important for the overall design:

- repository code can reason in terms of unit, status, time, relation and attribute constraints,
- search optimization and SQL strategy selection can be centralized in the adapter layer,
- PostgreSQL and DB2 differences can be isolated without duplicating the whole repository API.

5.12 GraphQL configuration and runtime

The Java GraphQL module is the other major part of the implementation. It has two responsibilities.

The first is configuration. As described in Chapter 4, the `Configurator` parses SDL, derives shapes for datatypes, attributes, records, templates and operations, reconciles those shapes with the repository catalog, and then builds the executable GraphQL schema.

The second is runtime execution. The central runtime class is `RuntimeService`. It keeps catalog metadata indexed in forms that are efficient for execution, wires GraphQL resolvers into the runtime, validates and stores raw units or domain payloads, loads units and payloads back from the repository, and delegates search and unit operations to specialized runtime services.

In practice, the Java GraphQL runtime sits directly on top of the repository layer. It does not bypass it. That keeps GraphQL behavior aligned with the same unit lifecycle, search semantics and catalog model used elsewhere in the Java implementation.

5.13 A typical Java-side flow

The most common end-to-end Java flow is conceptually straightforward:

1. obtain a repository instance from `RepositoryFactory`,
2. construct or load a unit,
3. store it through the repository layer,
4. search or retrieve it again through repository or GraphQL runtime services.

When a unit is stored from JSON, the repository reads the incoming JSON tree, updates status and unit name when applicable, pushes attributes into the unit model, and then persists the unit through the underlying storage path. When a search is executed, the repository builds a `UnitSearch` request around a

`SearchExpression`, delegates SQL generation and execution to the configured database adapter, and then resurrects matching units through the unit factory and cache layer.

So although the Java API surface looks object-oriented, the actual execution path is layered:

Repository API Owns unit-oriented behavior.

Search adapter Owns database-specific SQL for dynamic search.

Unit factory and cache Own reconstruction and reuse of persisted units.

Database procedures and schema Own the durable representation.

That layering is one of the main reasons the Java implementation serves well as the reference implementation: it makes the transition between high-level repository semantics and low-level persistent representation explicit.

5.14 Testing

Testing is split between unit tests in the Java modules and integration tests in `implementations/java/it`. The integration tests use real schema resources and real database-backed repository/GraphQL setup, which is important because much of IPTO's behavior emerges from the interaction between:

- the relational schema,
- the repository layer,
- the GraphQL SDL configuration,
- the runtime wiring.

5.15 Observations

The Java implementation is the reference implementation of IPTO today. It is where the repository semantics, catalog reconciliation behavior, GraphQL runtime wiring and SQL-backed search model are most fully expressed.

That also means the other implementation efforts are best documented relative to this one: either as parity-oriented ports, or as alternative runtime surfaces built on the same conceptual model.

6 Erlang Implementation

The repository also contains an Erlang implementation area under `implementations/erlang`. This is not just a placeholder. It is structured as an OTP application with repository, backend, GraphQL and HTTP modules. In the overall system structure it should still be understood as an alternative implementation surface rather than as the canonical definition of the data model. The canonical persistent model remains the database schema, and the canonical configuration model remains the GraphQL SDL and the repository catalog.

6.1 Overall structure

The Erlang implementation is organized around a layered module structure:

`ipto` Public facade API. This module exposes the main repository-facing functions and wraps calls in timing collection.

`ipto_repo` Central orchestration layer. This module coordinates unit helpers, persistence dispatch, search, relation and association helpers, lock handling and SDL-based GraphQL setup.

`ipto_backend` Behaviour definition for persistence backends.

`ipto_db` Backend dispatch point selecting the active implementation from application configuration.

`ipto_db_memory`, `ipto_db_pg`, `ipto_db_neo4j` Concrete backend implementations.

`ipto_graphql` and related modules GraphQL facade, SDL handling, resolver mapping and schema support.

`ipto_http` and related modules HTTP server and request handlers for GraphQL and health endpoints.

This is close in spirit to the Java split between facade, repository, backend-specific search/database logic and GraphQL runtime, but expressed in OTP style.

6.2 Facade and orchestration

The public entry module `ipto` is intentionally thin. It presents a surface close to the Java API and delegates actual work to `ipto_repo`. That means callers can work with functions such as:

- unit creation, retrieval and storage,
- search,
- relation and association management,
- lock management,
- status transitions,
- attribute and tenant lookup,
- GraphQL SDL inspection and configuration.

The actual coordination logic lives in `ipto_repo`. This module is the Erlang equivalent of a central repository service. It creates units, routes persistence through `ipto_db`, delegates relation and association work to dedicated helper modules, and applies SDL catalogs through the GraphQL setup path.

Search in the Erlang implementation includes parser support and symbolic query handling, but the textual-query surface is currently simpler than the Java implementation. It should be viewed as a practical operational query frontend rather than as full grammar-level parity with Java.

6.3 Backend contract

The persistence layer is explicitly abstracted behind the `ipto_backend` behaviour. The documented backend contract includes callbacks for:

- unit retrieval and existence checks,
- unit storage,
- search,
- relation and association mutation and lookup,
- lock handling,
- status updates,
- attribute and tenant metadata lookup,
- persistence of record and unit template metadata.

This is an architectural choice. Instead of scattering backend checks throughout the code, Erlang routes persistence through one behaviour-driven boundary and lets `ipto_db` dispatch to the selected backend implementation.

6.4 Backend implementations

The codebase currently defines three backend modes:

`memory` In-memory behavior for tests and lightweight runs.

`pg` PostgreSQL-backed behavior.

`neo4j` Neo4j-backed behavior.

PostgreSQL support prefers stored procedures for unit write/read and falls back to direct SQL if procedure calls fail. The Neo4j module implements the full callback surface of the backend contract.

This makes the Erlang implementation notable in one respect: compared to the Java implementation, which targets relational backends directly, the Erlang code is structured around backend plurality as a first-class concern.

6.5 GraphQL and HTTP surfaces

The Erlang implementation includes its own GraphQL subsystem:

`ipto_graphql` Public GraphQL facade.

`ipto_graphql_adapter` Adapter layer for `graphql_erl`.

`ipto_graphql_sdl` SDL inspection and validation support.

`ipto_graphql_schema` Schema definition for repository primitives.

`ipto_graphql_resolvers` and resource modules Resolver and operation mapping.

It also includes an HTTP layer with handlers for `/graphql` and `/health`, managed through supervised OTP components. This gives the Erlang implementation a self-hosted service shape, not just an embedded library shape.

6.6 Testing and completeness

The Erlang tree contains both ordinary EUnit tests and backend-specific integration tests, including:

- smoke tests,
- search parser tests,
- PostgreSQL integration tests,
- Neo4j integration tests,
- backend parity tests,
- GraphQL setup and execution tests,
- HTTP tests.

The implementation is still evolving, especially with respect to parity with Java search semantics and full persistence behavior.

6.7 Observations

Architecturally, the Erlang code is valuable for two reasons.

First, it makes the repository and GraphQL boundaries explicit in a way that is easy to compare with the Java and Rust implementations. Second, it treats backend substitution as a core design concern through the behaviour contract, which is useful when documenting the system as a whole.

7 Rust and Python

IPTO includes both a Rust implementation area and Python-facing entry points, and these two parts are intentionally connected. The Rust implementation exposes IPTO into a Python environment through `pyo3` and `maturin`.

7.1 Rust

The Rust code under `implementations/rust` is another implementation surface for the same conceptual model. It contains:

`src/repo.rs` Backend-agnostic repository orchestration.

`src/backend.rs` The backend trait that defines the implementation contract.

`src/backends/postgres.rs` The PostgreSQL backend.

`src/backends/neo4j.rs` The Neo4j backend.

`src/graphql.rs` and `src/graphql_sdl.rs` A Rust-side GraphQL runtime and SDL handling layer.

`src/lib.rs` The Python module entry point when the `python` feature is enabled.

The Rust implementation is *maturin-ready* and exposes a stable Python API via `pyo3`. This means the Rust code is not just an internal library. It is also the provider of the Python-visible IPTO surface.

7.2 PyO3 bridge

When the Rust crate is built with the `python` feature, `src/lib.rs` exposes a Python class named `PyIpto`. This class wraps the Rust repository service and backend implementations and exports repository and GraphQL operations into Python.

The Python-visible facade includes methods for:

- health checks and statistics,
- unit storage and retrieval,
- searches,
- status changes,
- lock handling,
- relation and association handling,
- attribute and tenant metadata lookup,
- GraphQL execution and SDL inspection/configuration.

This is the meaningful system-level connection between Rust and Python: Python is used as a host environment for the Rust implementation.

7.3 Build and packaging model

The Rust implementation is packaged for Python through `maturin`. The Rust project includes `Cargo.toml`, `pyproject.toml` and test/benchmark code that exercise the Python bridge. Python integration tests and soak benchmarks run through the `ipto_rust.PyIpto` module, so the PyO3 path is treated as a first-class supported interface.

7.4 Backend abstraction

As in the Erlang implementation, backend plurality is explicit in the Rust code. The trait `Backend` in `src/backend.rs` defines the storage contract for:

- unit retrieval, existence checks and storage,
- search,
- relations and associations,
- locks and status changes,
- attribute and tenant metadata,
- template persistence hooks,
- health and cache-control hooks.

This trait is intentionally repository-shaped rather than database-shaped. It does not expose raw SQL or driver-level concerns. That keeps `RepoService` as the main place where IPTO semantics are enforced while allowing PostgreSQL and Neo4j backends to diverge internally.

7.5 Repository service layer

The Rust orchestration layer lives in `src/repo.rs` as `RepoService`. It plays a role very similar to the Java `Repository` and the Erlang `ipto_repo` module.

In particular, `RepoService` is responsible for:

- delegating unit reads and writes to the selected backend,
- enforcing lifecycle rules around readonly state, locks and status transitions,
- composing native or service-level set-operation searches,
- exposing relation, association, attribute and tenant operations through a uniform surface,
- parsing and applying SDL-driven configuration.

One important design choice is that Rust does not rely only on backend-native search composition. If the backend reports that a native set-expression execution path is unsupported, `RepoService` can evaluate boolean set operations in the service layer and then page the merged results. That keeps the external behavior more stable across backends even when their native capabilities differ.

Rust also supports textual and symbolic search input, but that surface is currently simpler than the Java textual-query language. The Rust implementation is converging on Java semantics, yet the Java parser remains the more complete reference point for complex textual queries.

7.6 GraphQL runtime scope

The Rust implementation includes its own GraphQL runtime in `src/graphql.rs`. This runtime is not yet equivalent to the Java GraphQL runtime in breadth. It maintains a static registry of supported operations and dispatches repository-backed queries and mutations through the Rust service layer.

The runtime supports:

- read operations such as unit lookup, search, metadata lookup, health, lock state and relation/association reads,
- mutation operations such as status changes, unit storage, attribute creation, lock handling and relation/association updates,
- operation registration and allowlist filtering,
- GraphQL-style error envelopes for execution failures.

This is a compatibility-oriented runtime surface rather than a full clone of the Java GraphQL engine. It exposes a practical operational subset while broader parity is still being developed.

7.7 Python

Python serves two different roles:

Embedded consumer of Rust Through the `PyIpto` module exported by Rust via PyO3.

Operational tooling Through scripts that provision test environments, run integration tests, or execute backend benchmarks.

The second role is incidental from an architectural point of view. The first role is structural.

7.8 Current parity position

The Rust implementation status:

- repository core behavior is largely implemented,
- relations, associations, locking and status-transition surfaces are present,
- PostgreSQL and Neo4j backends are both active targets,
- the PyO3 surface is substantial rather than minimal,
- GraphQL support is present but still partial compared to Java,
- exact search and lifecycle parity still has edge-case gaps.

The Rust implementation is an active parity-seeking runtime, not an experimental side branch. It already carries meaningful repository behavior, a Python-facing integration surface, backend abstraction, and a growing GraphQL compatibility layer.

8 Admin Web

IPTO includes a web-based administration client under `apps/admin-web`. This application is not part of the conceptual core of IPTO in the same way as the database schema or the SDL configuration layer, but it gives administrators and developers a browser-based view of the repository.

8.1 Role in the system

The web client is a SvelteKit application for:

- schema administration,
- tenant-scoped tree browsing,
- unit search and result inspection.

This makes it a client of IPTO rather than a defining part of the persistence model. It sits on top of the repository and API layers described elsewhere in the manual.

8.2 Structure

The application is organized around a clear split between routes and reusable UI components:

`src/routes/+page.svelte` Overview page.

`src/routes/admin/+page.svelte` Administration views for attributes, records and templates.

`src/routes/trees/+page.svelte` Tenant-scoped unit tree browsing.

`src/routes/search/+page.svelte` Search UI, saved searches, filters, results and unit detail.

`src/lib/components` Shared UI components such as the header, lists, filters and detail panels.

`src/lib/api.js` The client-side API access layer.

8.3 Integration points

The application is structured to call backend HTTP endpoints rather than embedding repository logic locally. API access goes through `fetch(...)` calls in `src/lib/api.js`, which keeps the browser layer focused on interaction and presentation.

The integration surface includes operations related to:

- tenants,
- attributes,
- records,
- templates,
- saved searches,
- unit search and unit detail retrieval.

That is consistent with the role of the application: it is an operational and administrative frontend over repository-backed services.

8.4 Packaging

The web app can be built into the Quarkus static resources folder so that it is served by the same HTTP server as the Java backend. This is a useful deployment pattern because it reduces the number of independently deployed artifacts for the common case of a Java-based runtime.

8.5 Observations

The admin web is a practical systems interface: it exposes repository capabilities to a browser user, but it does so by relying on the backend models and services described in the rest of this manual.

9 Development Notes

This chapter summarizes the practical build, test and setup workflow that ties the different implementation areas together. It is intentionally operational. The subsystem chapters explain what the system is; this chapter explains how to develop or work with IPTO.

9.1 Repository layout

Development work is split across several implementation roots:

`implementations/java` Reference implementation and GraphQL runtime.

`implementations/erlang` OTP-based alternative implementation with PostgreSQL, Neo4j, GraphQL and HTTP support.

`implementations/rust` Rust implementation with PostgreSQL and Neo4j backends plus a Python bridge via PyO3.

`apps/admin-web` SvelteKit-based administration web client.

`shared/db` Shared schema, procedures and setup scripts for PostgreSQL and DB2.

`doc/system` The system manual itself.

9.2 Java build and test workflow

The Java build is Maven-based and centered on the parent project under `implementations/java/pom.xml`. The most useful commands are:

- `mvn -fimplementations/java/pom.xml -qtest` for Java unit tests across modules,
- `mvn -fimplementations/java/pom.xml -plrepotest` to focus on the repository module,
- `mvn -fimplementations/java/pom.xml -plgraphqltest` to focus on the GraphQL module,
- `mvn -fimplementations/java/pom.xml -plitverify` for integration tests.

The Java implementation expects a running database with schema and procedures already loaded when integration tests are executed.

9.3 Database provisioning

The shared database scripts under `shared/db/postgresql` and `shared/db/db2` are part of day-to-day development. In particular:

- `shared/db/postgresql/setup-for-test.py` provisions a local Docker-based PostgreSQL environment and loads schema, procedures and boot data,
- `shared/db/db2/setup-for-test.py` does the corresponding setup for DB2,
- `schema.sql`, `procedures.sql` and `boot.sql` define the persistent baseline used by multiple implementations.

These scripts are not merely test conveniences. They are the practical way to establish the canonical persistence environment used by Java integration tests and by backends in other language implementations.

9.4 Erlang workflow

The Erlang implementation uses `rebar3`. Useful commands include:

- `rebar3 compile`,
- `rebar3 eunit`,
- `rebar3 as graphql eunit` for GraphQL-enabled tests,
- `rebar3 as pg eunit` for PostgreSQL-backed integration tests when the required environment variables are set.

The Erlang implementation also supports runtime profiles for optional subsystems such as GraphQL and HTTP, along with release-oriented workflows including hot upgrade handling.

9.5 Rust and Python workflow

The Rust implementation is developed with both Cargo and Python tooling in mind. The local `Makefile` provides a compact workflow:

- `cargo test` for Rust-only tests,
- `make pg-up` and `make pg-test` for PostgreSQL-backed tests,
- `make neo4j-test` for Neo4j-backed tests,
- `maturin develop` for building the Python extension in development mode,
- `python3-mpytest-qpython_tests/test_ipto.py` for Python-facing tests of the PyO3 bridge.

This reflects the dual nature of the Rust implementation: it is both a Rust codebase and a Python-consumable package surface.

9.6 Admin web workflow

The administration web client under `apps/admin-web` is a SvelteKit application. The basic local workflow is:

- `npm install`
- `npm run dev`
- `npm run build`

It provides a minimal UI for administration, tree browsing and search. The build output can be packaged into the Quarkus application resources so the UI is served by the same HTTP server as the backend.

9.7 SDL-driven development

One recurring development pattern across implementations is SDL-first modeling. A GraphQL schema file is not just an API contract. In IPTO it also influences repository catalog setup. That means changes to SDL may require:

- validation of directives and type shapes,
- reconciliation against stored attributes, records and templates,
- database-backed tests to confirm the resulting catalog state,
- runtime checks in GraphQL-facing code.

For this reason, SDL evolution should be treated as a persistence-affecting change, not as a purely presentation-layer change.

9.8 Observations

The most important practical point is that IPTO is not developed as one isolated codebase. It is a coordinated system with a shared database model, a shared schema/configuration model, and several implementation surfaces. Good development practice therefore means verifying changes at the boundary they affect: database, catalog, SDL, runtime behavior, or language binding. The appendix in Section 11 collects the most common commands and variables used in that work.

10 Appendix: Developing Against IPTO from Java

This appendix describes a practical Java development flow against IPTO, based on the integration setup in `implementations/java/it` and in particular the test `GraphQLIT`. The important architectural point is that development does not begin by inventing ad hoc Java classes and queries. It begins with SDL and the repository catalog that SDL drives.

10.1 Start from SDL

In IPTO, the SDL does not only describe a GraphQL schema. It also describes the catalog attributes, record shapes, templates and runtime operations that the system will expose. In the integration tests, this is done by loading `implementations/java/it/src/test/resources/org/gautelis/ipto/it/schema2.graphqls` through `Configurator.load(...)`.

A representative excerpt from that SDL is:

```
type Beslut @record(attribute: beslut) {
    datum : DateTime @use(attribute: date)
    beslutsfattare : String
    beslutstyp : BeslutsTyp
    beslutsutfall : BeslutsUtfall
}

type Yrkan @record(attribute: yrkan) @template {
    beskrivning : String @use(attribute: description)
    beslut : Beslut
    producerade_resultat : [ProduceratResultat]
    person : Person
}

type Query {
    yrkan(id : UnitIdentification!) : Yrkan @ipto(operation: "LOAD_UNIT")
    yrkanden(filter: Filter!) : [Yrkan] @ipto(operation: "SEARCH")
    unitRaw(id : UnitIdentification!) : Bytes @ipto(operation: "LOAD_UNIT_RAW")
}
```

This SDL establishes several things at once:

- which attributes and records exist in the repository catalog,
- which record trees together form a unit template such as `Yrkan`,
- which fields are available through typed GraphQL retrieval,
- which repository operations are exposed as GraphQL queries or mutations.

The result is that the SDL constrains and enables what can be done through both the general Java API and the GraphQL API. The Java repository API is broader and can work directly with units and attributes, but the practical shape of those units still comes from the configured catalog and templates. The GraphQL interface is even more directly shaped by the SDL because its query and mutation fields are derived from it.

10.2 Bootstrapping repository and runtime

The integration setup follows the same pattern that an application would use:

1. create a repository through `RepositoryFactory.getRepository()`,
2. load the SDL through `Configurator.load(repo, reader, ...)`,

3. let the configurator reconcile attributes, records and templates into the catalog,
4. call `repo.sync()` so the repository sees the updated catalog state,
5. use the resulting `Repository` and `GraphQL` instances.

In simplified form, the setup looks like:

```
Repository repo = RepositoryFactory.getRepository();

try (InputStreamReader reader = new InputStreamReader(
        Objects.requireNonNull(getClass().getResourceAsStream("schema2.graphqls")))) {
    GraphQL graphQL = Configurator.load(repo, reader, wireOps, System.out).orElseThrow
    ();
    repo.sync();
}
```

That sequence is important. The SDL must be loaded before higher-level Java code assumes that attributes, records and templates exist in the repository catalog.

10.3 Working through the general Java API

Once the repository is configured, Java code can create and store units using the generic repository API. The test code in `GraphQLIT` shows the style clearly: it creates a unit, attaches record-valued and scalar attributes, and stores the result.

```
Repository repo = RepositoryFactory.getRepository();
Unit yrkan = repo.createUnit(tenantId, processId);

yrkan.withRecordAttribute("ffa:fysisk_person", person ->
    person.withNestedAttributeValue("ffa:personnummer", String.class,
        value -> value.add("19121212-1212")));

yrkan.withAttributeValue("dcterms:description", String.class,
    value -> value.add("Yrkan om vard av husdjur"));

yrkan.withRecordAttribute("ffa:beslut", beslut -> {
    beslut.withNestedAttributeValue("dcterms:date", Instant.class,
        datum -> datum.add(aSpecificInstant));
    beslut.withNestedAttributeValue("ffa:beslutsfattare", String.class,
        value -> value.add(aSpecificString));
});

repo.storeUnit(yrkan);
```

This API is generic rather than template-specific. There is no generated Java class for `Yrkan`. Instead, the code works with unit instances, attribute names and record attributes. The reason this remains usable is that the SDL has already established which attributes and record structures are meaningful. In other words, the Java API is general, but the configured catalog gives it a stable vocabulary.

For record-valued attributes, the repository API allows nested construction through `RecordAttribute`. This is how compound structures such as `Beslut`, `Ersattning` and `Period` are assembled without introducing one Java class per record type.

10.4 What the SDL changes for Java development

From a Java developer's perspective, changing the SDL can change development in several ways:

- adding or renaming `@attribute` entries changes which attribute names can be instantiated and searched,
- adding or reshaping `@record` types changes how nested record trees should be assembled,
- adding or reordering `@template` content changes which structures are treated as first-class unit templates,
- adding or removing `@ipto` operations changes which GraphQL operations are exposed and auto-wired.

This is why IPTO development should be understood as schema-driven even when the immediate client is Java code. The repository API does not disappear, but its useful operating space is defined by the SDL-backed catalog.

10.5 Using the GraphQL interface

The same configured system can then be addressed through GraphQL. The integration test shows three main patterns:

- typed retrieval of a configured unit shape,
- text-filtered search,
- raw payload retrieval and storage.

A typed retrieval looks like:

```
query Unit($id: UnitIdentification!) {
  yrkan(id: $id) {
    person {
      ... on FysiskPerson { personnummer }
      ... on JuridiskPerson { orgnummer }
    }
    beslut {
      datum
      beslutsfattare
    }
  }
}
```

The important point is that this query is possible because the SDL has defined:

- the `Yrkan` template,
- the nested `Beslut` and `Person` record shapes,
- the `yrkan(...)` query field,
- the operation binding `@ipto(operation: "LOAD_UNIT")`.

So GraphQL is not discovering the domain structure dynamically from stored rows. It is using the configured record and template shapes to present a typed API over the generic repository model.

10.6 Text search through GraphQL

GraphQL search in the integration test uses the `Filter` input object and the Java textual-query parser described earlier in this manual:

```
query Yrkan($filter: Filter!) {
  yrkanden(filter: $filter) {
    beslut {
      beslutsfattare
    }
  }
}
```

with variables conceptually shaped as:

```
Map.of(
    "filter", Map.of(
        "tenantId", 1,
        "where", "beslutsfattare = \"" + beslutsfattare + "\""
    )
)
```

The search expression string is parsed by the Java search subsystem, resolved against the configured attribute catalog, compiled into a `SearchExpression`, and then translated into SQL by the active database adapter. That means the GraphQL search surface is thin in the right way: it delegates search semantics to the same repository search model used elsewhere in Java.

10.7 Raw payload flows

The integration test also demonstrates that GraphQL can expose raw repository payloads as `Bytes`. This is useful when a caller needs to move generic unit JSON in and out of the repository without going through the typed GraphQL projection.

The configured SDL exposes both:

- raw reads such as `unitRaw(...)` and `unitsRaw(...)`,
- raw writes such as `lagraUnitRaw(...)` and the custom `lagraYrkandeRaw(...)` mutation.

For the raw-write case, the test base64-encodes a JSON unit and submits it through GraphQL. The configured mutation then delegates to the runtime service and ultimately to repository-backed storage. This shows that GraphQL in IPTO can act both as a typed domain projection and as a transport surface for generic repository payloads.

10.8 Custom operations versus built-in operations

The integration setup wires one custom mutation explicitly:

```
params.runtimeWiring().type("Mutation", t ->
    t.dataFetcher("lagraYrkandeRaw", env ->
        params.runtimeService().storeDomainRawUnit(tenantId, "Yrkan", bytes)));
```

This is the important distinction:

- many operations are standard repository/runtime operations selected by `@ipto(operation: ...)`,
- some operations can be custom-wired as GraphQL data fetchers when application-specific behavior is needed.

So the SDL determines the exposed operation surface, but Java code can still attach custom behavior at wiring time where the built-in runtime operations are not sufficient.

10.9 Practical development advice

For Java development against IPTO, the most reliable workflow is:

1. change the SDL first when the information model changes,
2. reload and reconcile the configuration into the repository catalog,
3. adapt Java repository code to use the new or changed attribute and record vocabulary,
4. adapt GraphQL queries or mutations only after the SDL and catalog are aligned,
5. verify the result with an integration-style test similar to `GraphQLIT`.

That sequence reflects the actual architecture. The Java API is broad and generic, but it becomes meaningful through the configured catalog. The GraphQL API is more explicitly shaped by the SDL, yet it still relies on the same repository and runtime services underneath.

11 Appendix: Key Commands and Environment Variables

This appendix gathers the most frequently used commands and configuration variables mentioned elsewhere in the manual.

11.1 Core commands

Area	Typical command
PostgreSQL test DB setup	`python shared/db/postgresql/setup-for-test.py`
DB2 test DB setup	`python shared/db/db2/setup-for-test.py`
Java unit tests	`mvn -f implementations/java/pom.xml -q test`
Java integration tests	`mvn -f implementations/java/pom.xml -pl it verify`
Erlang compile	`rebar3 compile`
Erlang tests	`rebar3 eunit`
Rust tests	`cargo test`
Rust Python tests	`python -m pytest -q python_tests/test_ipto.py`
Rust Python extension	`maturin develop`

11.2 Important PostgreSQL variables

The PostgreSQL-backed implementations reuse a common naming scheme for local development:

- `IPTO_PG_HOST`
- `IPTO_PG_PORT`
- `IPTO_PG_USER`
- `IPTO_PG_PASSWORD`
- `IPTO_PG_DATABASE`

In Java, the default JDBC configuration is instead read from `configuration_pg.xml`, but the practical target is the same local PostgreSQL instance used by tests and development tooling.

11.3 Important Neo4j variables

The Erlang and Rust implementations both use Neo4j-related environment variables. The common ones are:

- `IPTO_NEO4J_URL`
- `IPTO_NEO4J_DATABASE`
- `IPTO_NEO4J_USER`
- `IPTO_NEO4J_PASSWORD`

Additional tuning variables exist in the Erlang implementation for timeouts, retries and bootstrap behavior.

11.4 GraphQL SDL-related settings

Where the GraphQL layer is configurable at runtime, the most important settings are the ones that determine where SDL is read from. In the Erlang implementation, for example, this includes:

- app env `graphql_schema_sdl`,
- app env `graphql_schema_file`,
- env var `IPTO_GRAPHQL_SCHEMA_FILE`.

In the Java implementation, the concrete application packaging currently provides a GraphQL schema resource under `implementations/java/quarkus-app/src/main/resources/ipto.graphqls`.

11.5 Practical advice

When working on IPTO, the most reliable sequence is usually:

1. provision the database or backend environment first,
2. run the tests closest to the subsystem being changed,
3. if SDL or persistence semantics changed, run an integration-level check rather than only unit tests,
4. only then verify other implementation surfaces affected by the same conceptual change.

This is a direct consequence of the architecture described in the manual: many changes cross the boundaries between schema, catalog, runtime and language binding.

11.6 Example unit payload

The repository-facing JSON format used by the current implementations is generic. A minimal unit payload looks like this:

```
{
  "tenantid": 1,
  "corrid": "550e8400-e29b-41d4-a716-446655440000",
  "status": 30,
  "unitname": "example-unit",
  "attributes": [
    {
      "attrid": 101,
      "attrname": "dcterms:title",
      "attrtype": 1,
      "value": ["Example title"]
    }
  ]
}
```

The important points are:

- the unit header identifies tenant, correlation id, status and optional name,
- each attribute occurrence carries the repository attribute id and datatype id,
- primitive values are represented as arrays, even when a single value is present,

- nested records follow the same idea recursively, but use record-valued attributes and child attribute arrays.

This is the payload shape that is translated into `repo_unit_kernel`, `repo_unit_version`, `repo_attribute_v` and the vector tables during ingestion.